Lambda Calculus – in brief

Sebastian Mödersheim, DTU Compute

Version 0.2 of October 8, 2026

1 Introduction

This article was inspired by the style of the book "Theoretische Informatik – kurzgefasst" by Uwe Schöning that gives an introduction to the core topics of theoretical computer science with an emphasis on brevity: working with a minimal set of concepts and selected material, rather than being an encyclopedia. We felt something like this was missing for lambda calculus.

Starting in the 1930's, the lambda calculus was developed by Church, Rosser, Turing, Curry and many others, and forms the theoretical foundation of functional programming languages. The *pure* lambda calculus gives a mathematically appealing answer to question: what is a minimal set of concepts that is sufficient to describe all computable functions? Initially, there is only an *intuitive* notion of what computability means, i.e., what kind of description can be regarded is an algorithm. One can try to formalize this notion by defining a formal language for computation like Turing machines and lambda calculus or programming languages like C, Java, F#, Haskell. It turns out that these formalisms all have the same expressiveness: any function from input to output that can be computed by a program in one language can also be computed by a program in any other. That led Church and Turing to the thesis that these formalisms all describe exactly what our intuitive notion of computability is.

2 Syntax

The syntax of pure lambda calculus defines what are lambda expressions, and this consists of mainly three things: variables, function definition, and function application.

Definition 1 (Pure Lambda Calculus)**.** *Given a set of variables (i.e., identifiers), the set of* expressions *is defined as follows.*

- *If x is a variable then $\boxed{x}$ is an expression.*
- *If e is an expression and x is a variable, then also $\boxed{\lambda x.e}$ is an expression.*
- *If e and e' are expressions, then also $\boxed{e\ e'}$ is an expression.*
- *If e is an expression, then also $\boxed{(e)}$ is an expression.*

The intuitive understanding is as follows:[1]

- $\lambda x.e$ describes a function of an argument x and result e. The simplemost example is $\lambda x.x$ which takes an arbitrary value as input and just returns it, i.e., defining the identity function.
- $e\ e'$ means function application: the function that is defined by expression e is applied to the argument defined by the expression e'. For instance $(\lambda x.x)\ y$ can be reduced to y.

The syntax of pure lambda calculus can be written as a context-free grammar:

$$expr ::= var \mid \lambda var.expr \mid expr\ expr \mid (expr)$$

This grammar is ambiguous and we thus use the following conventions:

- $\lambda x.e_1\ e_2 = \lambda x.(e_1\ e_2)$, i.e., the scope of a function definition extends as far as possible to the right.
- $e_1\ e_2\ e_3 = (e_1\ e_2)\ e_3$, i.e., function application associates to the left.

The syntax is obviously the bare minumum for expressing functions: one needs at least function definition and function application, and in the body of a function definition one must at least be able to use the variable that the function receives as argument. Observe that we have neither built-in data types like integer numbers, nor built-in functions like integer multiplication, nor recursion. Surprisingly, this turns out to be sufficient to express all computable functions.

2.1 Example: Quantified Boolean Logic

As an example, we show how to express quantified boolean logic. We will give names to lambda expressions we define in the form

$$let\ f = e$$

where f is an identifier and e a lambda expressions. We can then use f in further expressions as an abbreviation, but (at least for now) we cannot make recursive definitions, i.e., f must not occur in e.

Let us first define the boolean constants true and false:

$$\begin{array}{lcl} let\ true & = & \lambda x.\lambda y.x \\ let\ false & = & \lambda x.\lambda y.y \end{array}$$

What may appear very strange is the fact that we define the *constants true* and *false* as some functions. This is because essentially in pure lambda calculus everything is a function. Each of the two takes as input an argument x and then returns another function that takes as input an argument y, and that function returns then either x or y depending on the represented truth value.

For instance, *true a b* (for two variables a and b) means $(\lambda x.\lambda y.x)\ a\ b$ which first reduces to $(\lambda y.a)\ b$ which reduces to a. Similarly *false a b* reduces to b. Let us thus in the following write instead of "$b\ e_1\ e_2$" the more easily readable notation

$$if\ b\ then\ e_1\ else\ e_2$$

[1]Note that $\lambda x.e$ corresponds to `fun x->e` in F# and `\ x.e` in Haskell. Function application, variables and parenthesis have identical syntax in F# and Haskell.

where b, e_1, e_2 are expressions. Our intention here is that b evaluates either to *true* or *false*, but pure lambda calculus is untyped, so there are no "guard rails" that prevent us from using expressions that do not intuitively make sense.

We can now define the Boolean connectives and quantifiers:

$$\begin{array}{lcl} \mathit{let\ not\ b} & = & \mathit{if\ b\ then\ false\ else\ true} \\ \mathit{let\ and\ b_1\ b_2} & = & \mathit{if\ b_1\ then\ b_2\ else\ false} \\ \mathit{let\ or\ b_1\ b_2} & = & \mathit{if\ b_1\ then\ true\ else\ b_2} \\ \mathit{let\ exists\ p} & = & \mathit{or\ (p\ true)\ (p\ false)} \\ \mathit{let\ forall\ p} & = & \mathit{and\ (p\ true)\ (p\ false)} \end{array}$$

The reader may check as an exercise that $\mathit{forall}\ \lambda x.\mathit{or}\ x\ (\mathit{not}\ x)$ evaluates to *true*.

3 Semantics

We have so far relied on the reader's intuition how to apply a function definition, but this needs to be made formal:

Definition 2 (β-Reduction)**.** *A term of the form $(\lambda x.e)\ e'$ is called a* β-redex.

A β-reduction *is of the form*

$$(\lambda x.e)\ e' \to e[x \mapsto e']$$

where $e[x \mapsto e']$ roughly means the term e where we replace every occurrence of x by e'.

Example: $(\lambda x.\lambda y.x)\ \mathit{true}\ \mathit{false}$
$\to (\lambda y.x)[x \mapsto \mathit{true}]\ \mathit{false} = (\lambda y.\mathit{true})\ \mathit{false}$
$\to \mathit{true}[y \mapsto \mathit{false}] = \mathit{true}$

The formal definition of $e[x \mapsto e']$ is actually a bit subtle. We need to prevent *capturing free variables*. For instance in $(\lambda x.\lambda y.x + y)\ y$ should *not* reduce to $\lambda y.y + y$ because there is a clash between two different things called y: the formal parameter of the inner lambda function and the free variable y that the outer lambda function is called with. We should rather rename the formal parameter y of the inner lambda function to a new name to avoid this clash: $(\lambda x.\lambda y.x + y)\ y \equiv (\lambda x.\lambda z.x + z)\ y \to \lambda z.y + z$

Here we have used a relation $\equiv$, called α-equivalence, which means renaming the formal parameter of a function. This does not make a difference for the meaning of a function.

Normal Form A computation means a sequence of reduction steps, denoted $\to^\star$ where the $\star$ indicates an arbitrary number of reduction steps $\to$ (including 0 reductions).

Definition 3 (Normal-Form)**.** *An expression e is in normal form, if no subterm of e is a* β*-redex.*

Example:

$(\lambda x.\lambda y.x)\ \mathit{true}\ \mathit{false}$
$\to (\lambda y.\mathit{true})\ \mathit{false}$
$\to \mathit{true} = \lambda x.\lambda y.x \quad \mathit{is\ in\ normal\ form}$

When we have reached a normal form, we consider that the result of the computation. However note that an expression may contain several redices, for instance:

$$\underbrace{(\lambda x.\lambda y.x)\overbrace{((\lambda z.z)\ true)}^{\text{redex}}}_{\text{redex}}$$

We have here the choice to first reduce the outer redex and obtain $\lambda y.(\lambda z.z)\ true$ and then reduce the inner redex to obtain the normal form $\lambda y.true$. Alternatively we could first reduce the inner redex to get $(\lambda x.\lambda y.x)\ true$ and then the outer to get to the same normal form. Thus in both evaluations we get the same normal form. The whole lambda calculus would not make sense, if this was not always the case, i.e., if a term to evaluate to different normal forms depending on the reduction strategy. A central theorem is thus that β-reduction in lambda calculus is indeed confluent:

Theorem 1 (Church-Rosser)**.** *β-reduction is confluent (modulo α renaming $\equiv$), i.e.,*

- *if $e \to^* e_1$ and $e \to^* e_2$*
- *and e_1 and e_2 are in normal form,*
- *then $e_1 \equiv e_2$.*

Thus, the normal forms are always equivalent up to α-renaming. We do not give a proof here.

Evaluation Strategy The evaluation strategy, i.e. which reduction is performed first, has no influence on the result of the computation as the Church-Rosser theorem says. However, the strategy can have an influence on whether we do actually get a result. As this is a bit complicated, we skip this here and choose a strategy that is in some sense optimal: if there is a reachable normal form, this strategy will reach it:

Definition 4. Outermost-leftmost *evaluation:*

- *The redex $(\lambda x.e_1)\ e_2 \to e_1[x \mapsto e_2]$ has priority over all redices in e_1 and e_2*
- *In $e_1\ e_2$ we first normalize e_1 and then e_2.*

4 Termination

Not every term has a normal form:

$$let\ e_\perp = (\lambda x.x\ x)\ (\lambda x.x\ x)$$

Intuitively this term does not really make sense, because x is both used as a function and as an argument to that function![2] However, pure lambda calculus is untyped, and writing such strange terms is perfectly legal. It holds that:

$$e_\perp = (\lambda x.x\ x)\ (\lambda x.x\ x) \to (x\ x)[x \mapsto (\lambda x.x\ x)] = (\lambda x.x\ x)\ (\lambda x.x\ x) = e_\perp$$

[2]This expression would thus not be allowed by the type system in typical functional programming languages, because the type constraints $x : \alpha \to \beta$ and $x : \alpha$ contradict each other.

Thus, $e_\perp \to e_\perp$ and we never reach a normal form.

One may wonder if it is a flaw of lambda calculus to allow for such terms and whether one should in principle forbid this. However note that in any reasonable programming language it is possible to define infinite loops like `while(true){}` in C or Java. Of course in this way there is no benefit from it, but syntactically restricting a language such that this is impossible to write would actually restrict the language as we see below. Moreover, we see below that we can use something similar to $e_\perp$ to expression recursion.

The Semantics Function We can now define the semantics, or meaning, of a lambda expression as its normal form, if it exists. Otherwise, its semantics is $\perp$ ("undefined"):

Definition 5. *For a lambda expression e its semantics $[\![e]\!]$ is either*

- *e' if $e \to^\star e'$ and e' in normal form,*
- *$\perp$ if there is no such e'.*

Thus $[\![\cdot]\!]$ is a partial function from expressions to expressions.

Note that by Theorem 1, if normal form e' exists, then it is unique (up to α-renaming).
Examples:

- $[\![(\lambda x.\lambda y.x)\ \mathit{true}\ \mathit{false}]\!] = \mathit{true}$
- $[\![(\lambda x.\lambda y.x)\ e_\perp\ \mathit{false}]\!] = \perp$ (undefined, does not terminate)
- $[\![(\lambda x.\lambda y.x)\ \mathit{true}\ e_\perp]\!] = \mathit{true}$

The Halting Problem The famous halting problem is the question for a given program and input for it, whether the program will come to a halt after finitely many steps (aka terminate). While that can be formulated and analyzed for any programming language, we develop it now for the lambda calculus: we ask whether a given lambda-expression e (representing both a program and an input for it) will evaluate to a normal form in finitely many steps:

Definition 6 (The Halting Problem)**.** $\mathit{Halt}(e) = \begin{cases} \mathit{true} & \text{if } [\![e]\!] \neq \perp \\ \mathit{false} & \text{otherwise} \end{cases}$

The question asked by Alan Turing in his 1936 paper *On computable numbers with an Application to the Entscheidungsproblem* is: can we write a program H that can correctly solve the halting problem, i.e., given any program e, can H analyze e and output *true* iff $\mathit{Halt}(e) = \mathit{true}$ and *false* otherwise? This is part of the more general question whether you can write *programs that analyze programs*. Turing showed that there are limits to this:

Theorem 2 (Turing)**.** *Halt is* undecidable*, i.e., there is no expression H such that $[\![H\ e]\!] = \mathit{Halt}(e)$ for every expression e.*

Proof. Suppose there is an expression H such that for every expression e, $[\![H\ e]\!] = \mathit{Halt}(e)$.
Consider the expression $\mathit{Test} = \lambda e.\mathit{if}\ H\ (e\ e)\ \mathit{then}\ e_\perp\ \mathit{else}\ \mathit{true}$
Then the following statements are equivalent:

- $Halt(Test\ \ Test) = true$
- $[\![Test\ \ Test]\!] \neq \bot$
- $[\![if\ \ H\ \ (Test\ \ Test)\ \ then\ \ e_\bot\ \ else\ \ true]\!] \neq \bot$
- $[\![H\ \ (Test\ \ Test)]\!] = false$
- $Halt(Test\ \ Test) = false$ q.e.a.

5 Syntactic Sugar

We now show that the pure Lambda calculus is sufficient to develop a full programming language (like F#) as *syntactic sugar*, i.e., as abbreviating syntax for a less easy to read expression in pure lambda calculus. This involves primitive data types like integers and arithmetic operations on them, pattern matching, recursion, and algebraic data types.

Note that this construction as syntactic sugar is helpful to understand constructs and logics, while for efficiency, actual interpreters and compilers for functional programming languages use whatever the underlying hardware offers.

5.1 Arithmetic

In the encoding of boolean logic in section 2.1 we had encoded the two constants *true* and *false* simply as functions that take two arguments and return either of them. We call this *continuation-style programming*: a boolean $b \in \{true, \texttt{false}\}$ takes two *continuations* c_t and c_f that say what the result is in case for each possible value of b, i.e., $true\ c_t\ c_f = c_t$ and $false\ c_t\ c_f = c_f$.

We can use a similar idea now for natural numbers, even though there are infinitely many of them: simply use the classical definition that a natural number is either zero or the successor (+1) of a natural number, in F# syntax:

$$\begin{array}{ll} type\ Nat & = zero \\ & \mid s\ of\ Nat \end{array}$$

Of course this is a horribly inefficient encoding for instance 5 is represented by the expression $s(s(s(s(s(zero)))))$; we see below how to make efficient encodings.

We can encode this data type by the following functions:

$$\begin{array}{ll} let\ zero & = \lambda x.\lambda y.x \\ let\ s\ n & = \lambda x.\lambda y.y\ n \end{array}$$

Like in the case of booleans, every natural number is represented as a function that takes as arguments two continuations, one for the case that the number is *zero*, and one that it is the s of a predecessor n; the difference is that the second continuation gets that predecessor n as an argument.

To easily work with these natural numbers, we now first define *pattern matching* for them as syntactic sugar; given arbitrary expressions e, e_0, and e_s,

we write $\boxed{\begin{array}{l} match\ e\ with \\ \mid 0\ \ \rightarrow e_0 \\ \mid s\ n \rightarrow e_s \end{array}}$ as nicer notation for $\boxed{e\ e_0\ (\lambda n.e_s)}$.

We do not have shown how we can express recursion in pure lambda calculus, but let us simply assume that we have it. Then we could now define addition as follows:

$$\begin{array}{l} \mathit{let\ rec\ add\ n\ m\ =} \\ \quad \mathit{match\ n\ with} \\ \quad \mid 0 \quad \to m \\ \quad \mid s\ n' \to s\ (\mathit{add}\ n'\ m) \end{array}$$

This is a direct encoding of the equations

$$0 + m = m \text{ and } s(n') + m = s(n' + m)$$

The reader may similarly define multiplication and exponentiation (assuming we have already recursion).

5.2 Recursion

We now show that in the pure Lambda calculus, we can indeed express recursion. Remember that $e_\perp = (\lambda x.x\ x)\ (\lambda x.x\ x)$ creates a (useless) infinite loop. But a similar construction can actually do something useful:

Definition 7 (Fixedpoint Combinator)**.**

$$\mathit{let}\ Y = \lambda F.\ (\lambda x.F\ (x\ x))\ (\lambda x.F\ (x\ x))$$

Here is a property that shows why Y is called a fixedpoint combinator. Let F be any expression, then it holds that:

$$\begin{array}{rcl} Y\ F & \to & (\lambda x.F\ (x\ x))\ (\lambda x.F\ (x\ x)) \\ & \to & F\ ((\lambda x.F\ (x\ x))\ (\lambda x.F\ (x\ x))) \\ & = & F\ (Y\ F) \end{array}$$

Thus: $Y\ F \to F(Y\ F) \to F(F(Y\ F)) \to \ldots$.

With that we can simulate recursion in the pure lambda calculus:

$$\begin{array}{l} \mathit{let\ add0}\ f\ n\ m\ =\ \mathit{match\ n\ with} \\ \quad \mid 0 \quad \to m \\ \quad \mid s\ n' \to s\ (f\ n'\ m) \\ \mathit{let\ add} = Y\ \mathit{add0} \end{array}$$

Observe that this definition contains no recursion, and yet it resembles very closely the recursive definition of *add* that we gave before. The idea is here that the basic function *add0* represents one recursion step, given that f already represents the recursive function. Then *add* is the *fixedpoint* of *add0*.

We can see that this construction works as follows. It holds for all expressions n, m:

$$\begin{array}{rcl} \boxed{\mathit{add}\ n\ m} & \to & Y\ \mathit{add0}\ n\ m \\ & \to & \mathit{add0}\ (Y\ \mathit{add0})\ n\ m \\ & = & \mathit{add0}\ \mathit{add}\ n\ m \\ & \to & \boxed{\begin{array}{l} \mathit{match\ n\ with} \\ \mid 0 \quad \to m \\ \mid s\ n' \to s\ (\mathit{add}\ n'\ m) \end{array}} \end{array}$$

More generally, suppose we have a recursive definition of a function f by an expression e that contains f:

$$let\ rec\ f = e$$

Then we can express this in pure lambda calculus simply as the following non-recursive definition (for a new symbol f_0):

$$let\ f_0 = \lambda f.e$$
$$let\ f = Y\ f_0$$

5.3 Algebraic Data Types

The same idea for encoding natural numbers we can also use to encode lists:

Algebraic Data Type	Encoding in λ-Calculus
$type\ \alpha\ List\ \ = Empty$	$let\ Empty\ \ = \lambda c_e.\lambda c_c.c_e$
$\quad\quad\quad